
PHYSIGYM: Bridging Agent-Based Biological Simulation and Reinforcement Learning — A State-Space Study on Tumor Microenvironment Control

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 This paper presents PHYSIGYM, a framework that integrates agent-based biological
2 simulation within standardised reinforcement learning environments. By connect-
3 ing the agent-based modelling framework PHYSICELL with the Gymnasium API,
4 we provide a flexible tool for exploring RL strategies that control *in silico* biologi-
5 cal processes. We demonstrate PHYSIGYM’s potential through a case study on a
6 simplified tumor microenvironment (TME) toy model, in which a Soft Actor-Critic
7 agent learns targeted drug-delivery policies that steer the microenvironment toward
8 an anti-tumoral state and, in the best episodes, achieve tumor elimination. We
9 further use this case study to systematically compare a family of observation spaces
10 — from a partially observed macrophage-count baseline, through spatial scalar
11 summaries, to full multi-channel spatial images. Training on a structured rectangle
12 layout and testing on a held-out network-field layout, we find that spatially ex-
13 plicit image observations generalize markedly better than scalar summaries: image
14 modes reach a mean held-out test return of $\approx +30$ to $+32$, whereas spatial-scalar
15 summaries train comparably ($+43$) but collapse to ≈ 0 out of distribution, and the
16 macrophage-only POMDP baseline fails to learn at all (-32). This demonstrates
17 that PHYSIGYM’s flexible state-space interface directly impacts policy quality and
18 generalization in spatially structured environments. PHYSIGYM is open-source at
19 <https://anonymous.4open.science/r/PhysiGym-76A7>.

20 1 Introduction

21 Biological systems are complex, dynamic, and often difficult to study in real life. In silico modeling
22 aims to mimic these systems, providing a controlled environment for exploring specific aspects of
23 biological processes and testing hypotheses. In particular, agent-based models (ABMs) provide
24 a powerful framework for simulating biological systems where agents (e.g. cells) interact dynam-
25 ically based on predefined rules. These models are used in oncology and immunology to study
26 emerging behaviors in complex biological systems, with the potential to propose novel therapeutic
27 strategies [Johnson et al., 2025, Laubenbacher et al., 2024, Rocha et al., 2024].

28 Ordinary differential equations (ODEs) and agent-based models (ABMs) are widely used to simulate
29 the tumor microenvironment (TME). ODEs provide compact system-level descriptions, while ABMs
30 capture spatial and cellular heterogeneity at higher computational cost. For dynamic treatment
31 regimes (DTRs) [Gray and Pallavicini, 1982], however, simulation alone is insufficient: designing
32 adaptive, sequential interventions requires a control framework. Reinforcement learning (RL), where
33 an RL agent learns policies through interaction with an environment, offers such a framework [Barto

et al., 1989]. By modeling treatment regimes as a sequence of decisions, RL enables adaptive control of biological simulations [Chakraborty and Murphy, 2014]. For efficient experimentation and policy learning, RL further requires a standardized interface. To address this, we introduce PHYSIGYM, an interface that connects PHYSICELL [Ghaffarizadeh et al., 2018], an ABM framework for multicellular simulations of tissues or cell cultures (tumor microenvironment, other disease tissues, or bacteria colonies), with Gymnasium [Towers et al., 2024], a widely adopted RL application programming interface (API). PHYSICELL enables flexible multiscale modeling of biological systems, coupled with a sophisticated underlying physics simulator (the BioFVM diffusion transport solver [Ghaffarizadeh et al., 2016]), while Gymnasium provides a structured framework for training and evaluating RL policies.

The first implementation of RL on an ABM framework based on BioFVM was proposed by Zade et al. [2020], which applied Q-learning [Watkins and Dayan, 1992] to optimize Temozolomide treatment regimes for patients with glioblastoma multiforme, a highly specific problem. More recently, Aif et al. [2025] used PHYSICELL as a complex (high-fidelity) simulation environment for tumor growth, combined with a simple (low-fidelity) RL training environment of coupled ordinary differential equations (ODE) to learn treatment strategies for solid tumors.

PHYSIGYM bridges PHYSICELL and Gymnasium to provide a reproducible and scalable framework for integrating agent-based simulations of biological systems with RL strategies. In the following, we present PHYSIGYM’s motivation and design, demonstrating how it directly connects ABMs and RL to control complex biological systems. In particular, we focus on the tumor microenvironment, a biological system made up of cancer cells and surrounding immune and stromal cells that engage in complex interactions and cross-talks, which is proven to be crucial for an effective response to immunotherapy, an important therapeutic approach in an increasing number of types of cancer.

To illustrate the framework, we build a simplified TME toy model and use it as an end-to-end RL case study. This model also serves as a controlled testbed for a practical question that arises in any spatially explicit biological environment: among the many observation spaces PHYSIGYM can expose (scalars, images, graphs), which ones help an RL agent learn an effective dynamic treatment regime? We find that image-based observations that preserve the spatial layout of cells and substrates outperform compact scalar summaries, both in cumulative return and in generalization to held-out spatial configurations. This is an application-level finding on a deliberately simplified model; the primary contribution of this work is PHYSIGYM itself.

Contributions. (1) PHYSIGYM: an open-source framework that bridges the PHYSICELL ABM and the Gymnasium RL API, turning any PHYSICELL model into an RL environment with minimal code changes. (2) A simplified TME toy model demonstrating an end-to-end RL case study in which an agent learns a spatial drug-delivery policy that drives the microenvironment toward an anti-tumoral state. (3) A systematic comparison of observation spaces on the toy model, showing that spatially explicit (image) representations help RL learn and generalize better than scalar summaries.

2 Methods

2.1 PHYSIGYM design and implementation

PHYSICELL is an open-source ABM framework written in C++ that models multicellular systems using classical mechanics and the BioFVM diffusive transport solver [Ghaffarizadeh et al., 2018, 2016]. Cells are agents governed by type-specific rule sets; substrates such as oxygen or cytokines diffuse through the domain. Intracellular signalling networks [Letort et al., 2018, Ponce-de Leon et al., 2023, Ruscone et al., 2024] and extracellular matrix fibres [Noël et al., 2024] can be added, yielding ABMs that are spatially explicit in 2D or 3D, off-lattice, and multiscale [Metzcar et al., 2019].

RL frames sequential decision-making as a Markov Decision Process (MDP) [Bellman, 1957, Puterman, 2014] with state space S , action space A , transition model T , and reward R . The objective is

$$\arg \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=0}^{\tau} \gamma^t r_t \mid s_0, \pi \right].$$

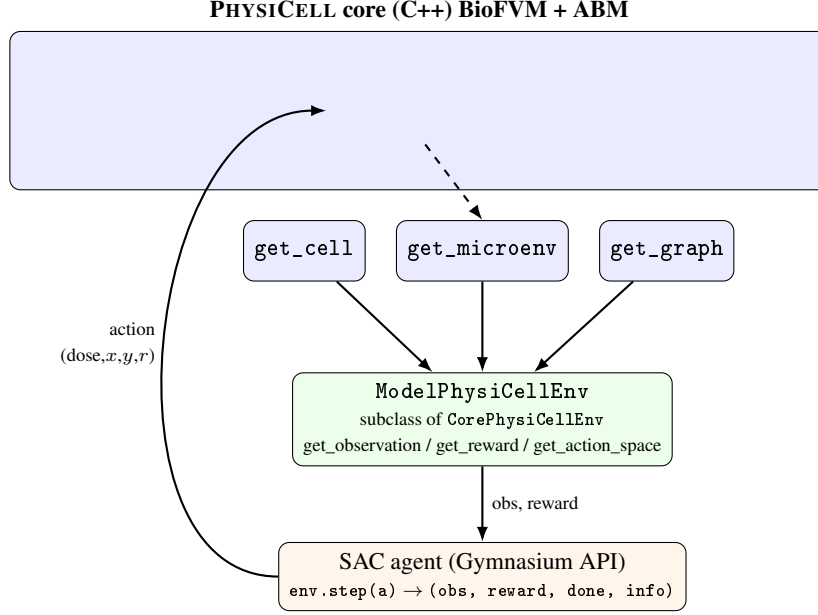


Figure 1: PHYSIGYM dataflow. The PHYSICELL C++ loop is exposed to Python as three callables (`physicell_start/step/stop`); only `physicell_step` is modified per model (here, to inject `drug_1`). Three read functions (`get_cell`, `get_microenv`, `get_graph`) surface cell positions, substrate fields, and neighbourhood graphs. A user subclasses `CorePhysiCellEnv` to assemble these into a Gymnasium observation/reward, and any RL algorithm drives the environment through the standard `step/reset` API.

83 Gymnasium [Towers et al., 2024] is the standard Python interface for MDPs, promoting modularity
 84 and code reuse across algorithms and environments.

85 PHYSIGYM connects these two ecosystems by extending the Python interpreter [van Rossum, 1995,
 86 Python Software Foundation, 2024] with PHYSICELL. The C++ simulation loop is split into three
 87 Python-callable functions: `physicell_start`, `physicell_step`, and `physicell_stop`. For most
 88 models only `physicell_step` needs to be modified (e.g. to inject a drug). Three read functions
 89 `get_cell`, `get_microenv`, and `get_graph` — return cell positions, substrate concentration fields,
 90 and cell neighbourhood graphs respectively; all are implemented in `physicellmodule.cpp`.

91 The interface between PHYSICELL and Gymnasium uses PHYSICELL’s built-in *user parameter*
 92 and *custom data* constructs, configurable through PhysiCell Studio [Heiland et al., 2024]
 93 without recompilation. Users subclass `CorePhysiCellEnv` in `physicell_model.py` to spec-
 94 ify observation space, action space, and reward; the base class handles all model-independent
 95 Gymnasium bookkeeping. Any existing PHYSICELL model can therefore become a PHYSI-
 96 GYM environment with minimal code changes. PHYSIGYM is tested on Linux, WSL, and
 97 macOS, with continuous integration via GitHub Actions. Documentation and tutorials are at
 98 <https://anonymous.4open.science/r/PhysiGym-76A7/man>.

99 Figure 1 shows the resulting dataflow, and Listing 2 gives a complete minimal environment: a user
 100 overrides only the observation, action, and reward, while the base class handles the PHYSICELL
 101 start/step/stop loop and all Gymnasium bookkeeping.

102 2.2 Implementation of a Tumor Microenvironment Toy Model

103 We developed a simplified tumor microenvironment (TME) *toy model* to illustrate the potential uses
 104 of PHYSIGYM. The model is deliberately small and computationally cheap so that we can train RL
 105 agents to convergence many times over (across observation spaces and seeds), while still retaining the
 106 spatial, multicellular, and chemically coupled structure that distinguishes agent-based models (ABMs)
 107 from ODE surrogates. It includes three cell types, five diffusible substrates, and a set of simple

```

from physigym.envs import CorePhysiCellEnv
from gymnasium import spaces
import numpy as np

class ModelPhysiCellEnv(CorePhysiCellEnv):
    # Action: localized drug injection (dose, x, y, radius), all normalized.
    def get_action_space(self):
        box = lambda: spaces.Box(0.0, 1.0, (1,), np.float32)
        return spaces.Dict({"drug_1_dose": box(), "drug_1_x": box(),
                           "drug_1_y": box(), "drug_1_radius": box()})

    # Observation: e.g. per-cell-type 64x64 image channels (img_mc_cells).
    def get_observation_space(self):
        return spaces.Box(0, 255, (self.n_channels, 64, 64), np.uint8)

    def get_observation(self):
        # get_cell / get_microenv surface ABH state from the C++ core.
        return self.render_cell_channels() # -> (C, 64, 64) uint8

    # Reward: tumor suppression minus a drug-cost penalty (Eq. in paper).
    def get_reward(self):
        dc = (self.count_prev - self.count) / self.growth_norm
        return self.w_cell * dc - self.w_dose * self.get_dose_spent()

    def get_terminated(self):
        return self.count == 0 # tumor eradicated

```

Figure 2: A complete PHYSIGYM environment: subclass CorePhysiCellEnv and override the observation, action, and reward. The base class handles all model-independent Gymnasium book-keeping (reset/step, the PHYSICELL start/step/stop loop, and the get_cell/get_microenv read-back), so no C++ recompilation is needed to expose a new state space. (Condensed from physicell_model.py.)

108 agent-based rules. This toy model highlights how the RL framework can control tumor progression
109 by modulating the whole microenvironment, rather than solely targeting cancer-cell killing.

110 **Domain and Dynamics.** The continuous-space environment is simulated on a $64 \times 64 \mu\text{m}$ 2D
111 domain. The RL gym step interval is $\Delta t = 15$ min, with episodes running up to 10,080 min (7
112 simulated days).

113 **Entity Dynamics & The Control Challenge.** The environment formulates a highly non-linear,
114 indirect control problem. The agent’s action (deploying a localized therapeutic, drug_1) has zero
115 direct cytotoxicity. Instead, the agent must learn to manipulate the environment’s underlying transition
116 dynamics by reversing a spatial immunosuppression loop. The system comprises three interacting
117 cellular populations whose movements are governed by biochemical spatial gradients (chemotaxis):

- 118 • **Tumor Cells (The Target):** Immobile entities that continuously divide. They actively
119 modify the environment by secreting a chemical signal (tumor_molecule) that suppresses
120 the local immune system.
- 121 • **Macrophages (The Mediators):** Mobile, immortal entities that navigate toward the tumor.
122 They exhibit state-dependent behavior: the tumor’s chemical signal successfully “hijacks”
123 them, transforming them from an anti-tumoral to a pro-tumoral state.
- 124 • **T-cells (The Effectors):** Mobile, immortal entities capable of triggering tumor apopto-
125 sis. However, their spatial navigation is strongly repelled by the presence of pro-tumoral
126 macrophages.

127 This setup creates a hostile, self-reinforcing feedback loop: the tumor protects itself by converting
128 macrophages, which in turn dynamically block T-cells from reaching the target. As illustrated in
129 Figure 3, the RL agent must maximize reward by learning to precisely deploy its drug to “re-program”
130 the macrophages back to an anti-tumoral state. This spatial intervention remodels the chemical
131 gradients, clearing a path for T-cells to navigate to the tumor and execute their killing function.

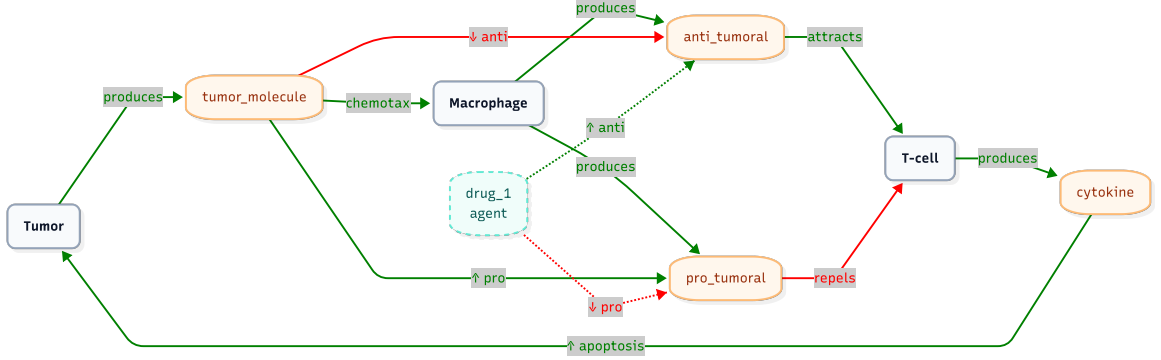


Figure 3: Tumor Microenvironment (TME) signalling network. Green solid arrows denote production, activation, or attraction, while red solid arrows indicate suppression or repulsion. The dashed pathways represent the intervention of the agent-controlled drug (drug_1), highlighting its role as an immune-modulator that repolarizes macrophages to indirectly drive T-cell mediated tumor apoptosis.

2.3 Implementation of an RL approach to control the TME

Our overall objective is to achieve complete tumor eradication while minimizing drug exposure. We describe the action space A , the reward R , and the training and test distributions. The observation space S is the component we vary across experiments: in Section 3 we compare a family of observation modes — spanning a partially observed scalar baseline, spatial scalar summaries, and multi-channel spatial images — to assess whether spatially explicit representations help the agent learn and generalize on this toy model.

Action space. At each gym step the agent selects $a_t = (\text{dose}, x, y, r) \in [0, 1]^4$, where dose is the drug concentration applied, (x, y) is the injection centre (normalised domain coordinates), and r is the injection radius (normalised by the domain diagonal).

Reward. The reward is a normalised tumor-suppression signal minus a weighted drug-cost penalty:

$$r_t = w_{\text{cell}} \frac{c_{t-1} - c_t}{c_{t-1}(e^{\lambda \Delta t} - 1)} - w_{\text{dose}} d_t, \quad (1)$$

where c_t is the alive tumor-cell count at step t , λ is the intrinsic growth rate (from `PhysiCell_settings.xml`), d_t is the normalised dose spent, $w_{\text{cell}} = 0.3$, and $w_{\text{dose}} = 2$. The denominator $c_{t-1}(e^{\lambda \Delta t} - 1)$ is the number of new tumor cells expected over one step from unchecked intrinsic exponential growth, so the ratio measures the observed change *relative to* the growth the agent had to overcome. This makes the tumor term scale-free in the absolute cell count and comparable across initial conditions with different tumor sizes: a value of $+1$ means one step’s worth of expected growth was fully cancelled, independent of how many cells were present. Raising the dose weight from 1 to 2 relative to earlier experiments strengthens the pressure to spend drug sparingly, so the agent is rewarded for converting each unit of dose into effective macrophage repolarisation rather than for blanket dosing. The d_t term penalises drug use directly, keeping the learning signal clean without conflating temporal dosing strategy with total exposure.

Training and Test Distributions

To ensure the model learns robust policies rather than overfitting to specific spatial configurations, we train and test on two *structurally distinct* spatial distributions, so that test performance measures out-of-distribution (OOD) generalisation rather than memorisation of the training geometry:

- **Rectangle (training):** Each cell type is placed uniformly in a narrow vertical strip, with non-overlapping strip positions (Appendix A). This is a highly structured, low-entropy layout.
- **Network-field (held-out test):** Initial positions are sampled from a spatially correlated random field (Appendix A), producing irregular, clustered configurations never seen during training.

164 Training on the simple, regular rectangle layout and testing on the richer, irregular network-field is a
 165 demanding transfer: a policy that merely exploits the fixed strip geometry of the rectangle cannot
 166 solve the network-field, so the test return isolates whether an observation space encodes *transferable*
 167 structure. Figure 8 illustrates representative examples of both distributions.

168 2.4 State space S : observation spaces compared

169 In a tumor microenvironment there are different cell types and different molecules/chemical com-
 170 pounds (substrates). The simulator can expose several kinds of data: some carry explicit spatial
 171 information, others are spaceless summaries (e.g. a small vector of global statistics). Because PHYSI-
 172 GYM can build all of these from the same underlying simulation, we use the toy model to study how
 173 the choice of observation space affects the dynamic treatment regime that RL learns, comparing nine
 174 modes on cumulative return and on generalization to unseen spatial configurations.

175 We compare observation modes spanning a spectrum from a partially observed scalar baseline to
 176 full multi-channel spatial images. Let $N_c = 3$ (tumor, T-cell, macrophage) and $N_s = 5$ substrates.
 177 Several modes optionally split the macrophage population into its $M1$ (anti-tumoral) and $M2$ (pro-
 178 tumoral) polarisation states, which we denote with the suffix `m1m2`; the classification rule is given
 179 below.

180 **Macrophage polarisation (M1/M2).** Macrophages do not carry an explicit polarisation label in
 181 the simulator; their state is implicit in the local chemical environment. We recover it from the
 182 microenvironment: for each alive macrophage we look up the two competing substrate fields at its
 183 nearest voxel and classify it as $M2$ (pro-tumoral) if $\text{conc}_{\text{pro_tumoral}} > \text{conc}_{\text{anti_tumoral}}$ at that voxel,
 184 and $M1$ (anti-tumoral) otherwise. This makes the pro-/anti-tumoral balance — the very quantity the
 185 agent must flip — observable as a count (scalars modes) or as two spatial channels (`img` modes),
 186 rather than leaving it latent in the substrate fields.

187 Scalar Observation Spaces

188 **scalars_macrophages (POMDP baseline):** Alive count per cell type, with the macrophage slot
 189 split into M1 and M2 counts; each is normalised to the initial population ($s = c/c_{\text{init}} - 1$). This mode
 190 exposes only *how many* macrophages are polarised each way and carries no spatial information at all
 191 — neither cell positions nor substrate maps. It is therefore a strongly partially observed (POMDP)
 192 baseline: the true state that governs the transition dynamics (where cells and gradients are) is hidden,
 193 and the agent must act on an aggressively compressed summary.
 194 *Space:* $\mathbb{R}^{N_c+1}$

195 **spatial_scalars_cells_substrates:** Per-type normalised cell counts and per-substrate max-
 196 ima, augmented with 6 k -means spatial statistics per cell type (cluster occupancy and centroids),
 197 giving a compact spatial summary without full images.
 198 *Space:* $\mathbb{R}^{N_c+N_s+6kN_c}$

199 **spatial_scalars_cells_m1m2 / ..._substrates_m1m2:** As above but with the macrophage
 200 population split into M1/M2 for both the scalar counts and the k -means spatial features
 201 (`..._substrates_m1m2` additionally keeps the substrate maxima).
 202 *Space:* $\mathbb{R}^{(N_c+1)+[N_s]+6k(N_c+1)}$

203 **spatial_scalars_cells_spatial_no_scalars_substrates_m1m2:** M1/M2 cell counts
 204 plus k -means spatial features for *both* cell types and substrate fields (spatially resolved substrate
 205 summaries in place of scalar maxima).
 206 *Space:* $\mathbb{R}^{(N_c+1)+6kN_s+6k(N_c+1)}$

207 Image-Based Observation Spaces

208 **img_mc_cells (I1):** One 64×64 channel per cell type.
 209 *Space:* $\mathbb{R}^{N_c \times 64 \times 64}$ (uint8)

210 `img_mc_cells_m1m2`: I1 with the macrophage channel split into separate M1 and M2 spatial
 211 maps.
 212 *Space*: $\mathbb{R}^{(N_c+1) \times 64 \times 64}$ (uint8)

213 `img_mc_cells_substrates (I2)`: I1 concatenated with all five substrate concentration maps,
 214 providing direct spatial feedback on drug distribution and the polarising factors.
 215 *Space*: $\mathbb{R}^{(N_c+N_s) \times 64 \times 64}$ (uint8)

216 `img_mc_cells_substrates_m1m2 (full information)`: I2 with the additional M1/M2
 217 macrophage channels. This mode carries every signal the others expose — cell positions, substrate
 218 fields, and the explicit polarisation split — and serves as the near-fully-observed upper reference
 219 against which the compressed modes are compared.
 220 *Space*: $\mathbb{R}^{(N_c+N_s+2) \times 64 \times 64}$ (uint8)

221 An observability spectrum

222 Together these modes trace an observability spectrum, from `scalars_macrophages` — a POMDP
 223 that sees only polarisation counts — to `img_mc_cells_substrates_m1m2`, which spatially resolves
 224 every state variable relevant to the control problem. The indirect drug mechanism (Section 2.2)
 225 requires multi-step credit assignment: the agent injects drug, macrophages repolarise over several
 226 steps, anti-tumoral gradients shift, T-cells migrate, and only then do tumor cells die. Scalar summaries
 227 cannot tell the agent whether an injection reached macrophages, whether polarisation occurred, or
 228 where T-cells will concentrate next; only the per-pixel substrate and M1/M2 channels of the image
 229 modes make the polarisation state and drug distribution spatially legible at each step.

230 3 Results

231 3.1 Aggregated performance

232 Table 1 summarises final performance (seeds averaged). Figures 4 and 6 show learning curves vs.
 233 cumulative environment steps with mean $\pm$ std shading across seeds.

Table 1: Performance at end of training (train = rectangle, test = network-field, seeds averaged). $\text{Train}_\mu/\text{Test}_\mu$: mean EWMA-50 return. $\text{Train}_\sigma/\text{Test}_\sigma$: std across seeds; n = number of seeds. This is the curated main set; the remaining m1m2 scalar variants appear in Appendix C, and 95% bootstrap confidence intervals on the mean in Appendix D. RAND is a random-action policy (5 seeds), included as a learning-free reference: modes below it fail to beat random behaviour.

ID	Mode	n	Train_μ	Train_σ	Test_μ	Test_σ
I1	<code>img_mc_cells</code>	5	+30.8	9.5	+33.4	11.3
I2	<code>img_mc_cells_substrates</code>	5	+45.3	5.0	+32.7	4.7
I2m	<code>img_mc_cells_substrates_m1m2</code>	5	+40.8	8.5	+32.5	9.4
S3s	<code>spatial_scalars_cells_substrates</code>	5	+42.9	18.8	−0.0	9.1
RAND	random policy (baseline)	5	−18.4	4.0	−25.3	3.0
POMDP	<code>scalars_macrophages</code>	5	−20.4	8.3	−31.8	2.3

234 3.2 Key findings

235 **Generalization, not training fit, separates the modes.** On the training rectangle layout the spatial-
 236 scalar summary S3s is fully competitive — its training return (+42.9) is on par with the best image
 237 mode I2 (+45.3). The distinction appears only out of distribution: on the held-out network-field, the
 238 three image modes retain $\approx +32$ to $+33$ test return, while S3s collapses to -0.0 . Training fit alone
 239 would have ranked the scalar summary among the best; only the OOD test reveals that it has overfit
 240 the fixed strip geometry of the rectangle. This separation is statistically supported: the image modes’
 241 95% bootstrap confidence intervals on the mean test return lie entirely above $+23$ and do not overlap
 242 those of any spatial-scalar mode, all of which contain zero (Appendix D). Pooling the per-seed test
 243 returns of the three image modes against the four spatial-scalar modes, the image family is higher

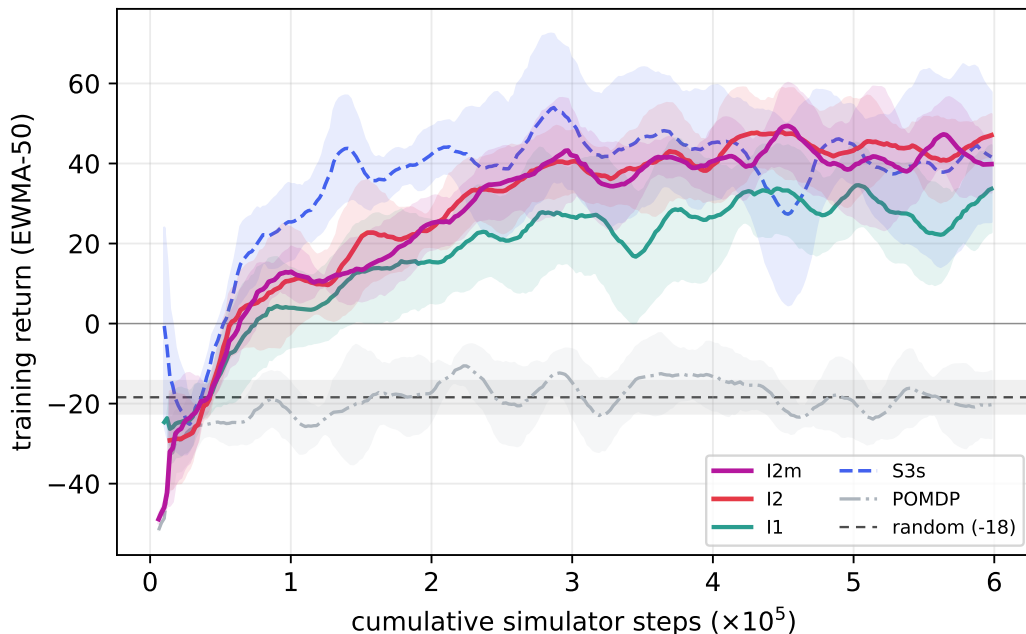


Figure 4: Training return on the rectangle layout (EWMA-50, mean $\pm 1\sigma$ across seeds) vs. cumulative *simulator* steps ($\times 10^5$; each of the 10^5 agent decisions is held for 6 simulator steps, giving a 6×10^5 budget). All modes learn on the training layout except the `scalars_macrophages` POMDP baseline, which flatlines near -20 . Spatial-scalar summaries (S3s) train as fast as, or faster than, the image modes here — the separation only appears out of distribution (Figure 6).

244 with a Mann–Whitney U -test $p = 6 \times 10^{-7}$ and a Cliff’s $\delta = +0.97$ (near-complete rank separation),
 245 so the gap is not an artefact of seed selection (Appendix E).

246 **Spatially explicit observations transfer; scalar summaries do not.** All three image modes —
 247 `img_mc_cells` (I1, $+33.4$), `img_mc_cells_substrates` (I2, $+32.7$), and the full-information
 248 `img_mc_cells_substrates_m1m2` (I2m, $+32.5$) — generalize to the network-field, and the gaps
 249 among them are within one seed-standard-deviation. That I1 already generalizes well indicates that,
 250 in this toy model, preserving the *spatial layout* of cells is the primary driver of transfer; the extra
 251 substrate and M1/M2 channels of I2/I2m do not further separate on average, though I2m gives the
 252 single best test return.

253 **The POMDP baseline is no better than acting randomly.** `scalars_macrophages`, which
 254 observes only M1/M2 counts and no spatial information, fails to improve on either distribution
 255 (train -20.4 , test -31.8) — and, crucially, does *not* beat a random-action policy (train -18.4 , test
 256 -25.3 ; RAND in Table 1). Its test return is if anything below random, and the two bootstrap CIs
 257 barely overlap (Appendix D): the macrophage-count signal is not merely uninformative but actively
 258 misleading, steering dosing worse than chance. Knowing *how many* macrophages are polarised
 259 without knowing *where* the drug, cells, and gradients are is insufficient to solve the indirect, spatially
 260 mediated control problem — a concrete illustration of how observation-space choice can render an
 261 otherwise learnable task effectively unlearnable. By contrast, every spatial-scalar mode, though it
 262 fails to generalize, still clears the random baseline on test (≥ 0 vs. -25.3), confirming they learn
 263 *some* transferable structure — just far less than the image modes.

264 **On seed robustness.** We do not observe a systematic seed-variance advantage for any family on
 265 this benchmark: end-of-training test std is 5–11 for the image modes and 9 for S3s (Table 1, Figure 5),
 266 and a Levene test on the across-seed test-return spread of the image versus spatial-scalar families is
 267 not significant ($W = 0.67$, $p = 0.42$), so we cannot reject equal dispersion. The POMDP baseline’s
 268 low std (2.3) reflects consistent failure rather than robustness.

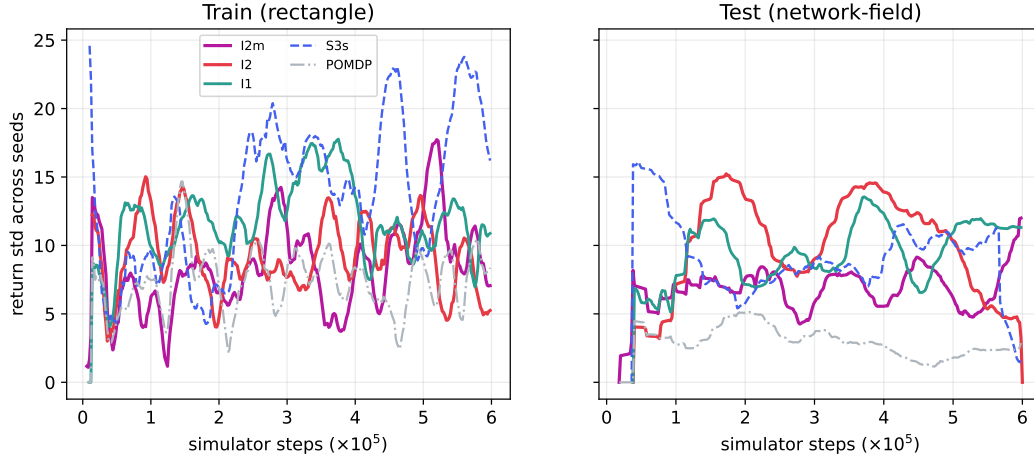


Figure 5: Standard deviation of return across seeds vs. simulator steps: training on rectangle (left) and test on network-field (right). Inter-seed variability is broadly comparable across the learning modes and fluctuates over training, so we do not claim a systematic variance advantage for any single family; the POMDP baseline has the lowest test variance simply because it converges to a uniformly poor policy. At end of training the image modes reach test return $\approx +30$ with std $\approx 7\text{--}12$ (Table 1).

3.3 The role of the M1/M2 split and spatial scalars

The full mode family (Appendix C, Table 4) lets us isolate the effect of the explicit M1/M2 polarisation split. Adding it to the cell image (I1 $\rightarrow$ img_mc_cells_m1m2) does not improve average OOD test return ($+33.4 \rightarrow +26.0$; the difference is not significant, Welch $p = 0.40$ at $n = 5$), and adding it on top of the full image (I2 $\rightarrow$ I2m) leaves it essentially unchanged ($+32.7 \rightarrow +32.5$). We stop short of calling the split *redundant*: with only five seeds an equivalence test (TOST, margin $\pm 10 \approx 1\sigma$) does not confirm equivalence for either pair, so the honest statement is that the pre-computed split yields *no detectable improvement* on this toy model rather than a proven null effect (Appendix E). A plausible mechanism is that, once the substrate fields are spatially available, the CNN can already read out polarisation from the pro_tumoral/anti_tumoral channels. Among the spatial-scalar modes, none generalize (all within $[-0.0, +9.5]$ test return) regardless of whether they carry the M1/M2 split or spatially resolved substrate summaries — the common failure is the loss of per-cell spatial layout, which the k -means summaries do not preserve.

Appendix F shows a matched example episode comparing an image mode, a spatial-scalar mode, and the POMDP baseline rolled out from an identical network-field initial condition, illustrating the failure mode that summarises the comparison: without a spatially explicit representation the agent cannot target the drug precisely enough to convert dose into effective macrophage repolarisation on an unseen layout — the image mode reaches near-eradication (3 tumor cells) while the scalar and POMDP modes leave 70 and 130 cells respectively.

4 Discussion

The central outcome of this work is PHYSIGYM itself: a framework that lets RL agents control a full agent-based TME through a standardised interface, without recompilation, and with arbitrary observation and action spaces. The toy-model case study shows the framework end-to-end — an agent learns a spatial drug-delivery policy that steers the microenvironment toward an anti-tumoral state and, in the best episodes, eradicates the tumor.

The toy model also produces a clear result on observation spaces, and crucially one that only surfaces under distribution shift. On the training rectangle layout, spatial-scalar summaries are fully competitive with image observations; it is on the held-out network-field that spatially explicit (image) observations pull ahead, generalizing to $\approx +30$ test return while the scalar summaries collapse to ≈ 0 . The CNN in the image modes can compute cross-channel co-localisation statistics (e.g. whether drug_1 and macrophages overlap at the same location) directly from the pixel layout statistics

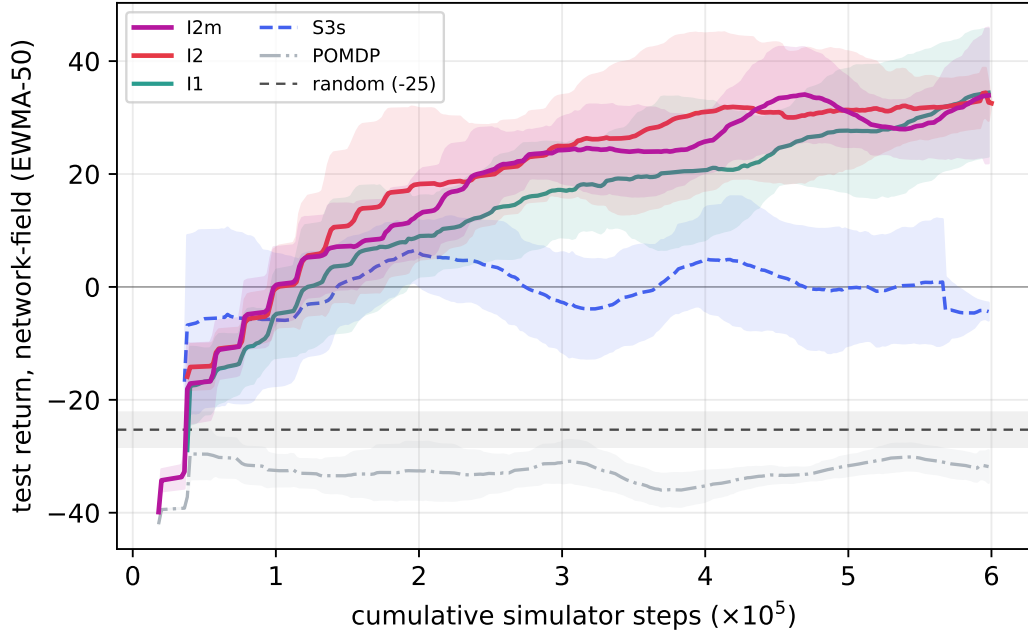


Figure 6: Held-out test return on the network-field layout (mean $\pm 1\sigma$ across seeds, EWMA-50) vs. cumulative simulator steps. The image modes (I2m, I2, I1) climb steadily to $\approx +30$, whereas the spatial-scalar summary (S3s) — which trained *best* (Figure 4) — collapses to ≈ 0 , revealing that it overfits the fixed rectangle geometry rather than learning transferable structure. The dashed line marks the random-policy baseline (test ≈ -25 , shaded $\pm 1\sigma$); the scalars_macrophages POMDP baseline tracks at or below it throughout, i.e. never beats random. Generalization, not training fit, is where spatially explicit observations separate from scalar summaries.

that transfer across geometries — whereas scalar modes that encode k -means cluster centroids and absolute positions overfit the fixed strip geometry of the training layout. Notably, once cell layout is spatially available, preserving the substrate fields (I2) or pre-computing the M1/M2 split (I2m) does not further improve average transfer over the cell-only image (I1) on this toy model, though the full-information mode gives the single best test return.

The overfit is a drug-aiming failure, measurable in the actions. The claim above is not merely architectural: it is visible directly in the policies’ behaviour. For every logged episode we compute the mean distance between the drug-injection centre (`action_x`, `action_y`) and the tumour centroid (from the episode initial conditions), averaged over the steps on which a dose is actually applied, and compare this aiming error between the training rectangle and the held-out network-field (Fig. 7). On the training layout all six modes aim equally well (mean normalised error ≈ 0.34 – 0.36 for both families). Under the geometry shift, image modes hold their aim or slightly improve it (I2: $0.359 \rightarrow 0.343$; I2m: $0.352 \rightarrow 0.333$), whereas every scalar mode’s aim *degrades* (S3s: $0.341 \rightarrow 0.414$; S3m: $0.348 \rightarrow 0.406$) and its step-to-step variance roughly doubles. In other words, the scalar policies learn *where* the rectangle tumour tends to sit rather than *how to find* the tumour, so when the network-field moves it they inject into empty tissue; the median final tumour count on test rises accordingly (image ≈ 73 – 99 vs. scalar 107 – 136). This is the concrete mechanism behind the aggregate return gap: the generalisation failure is an aiming failure induced by position-absolute features.

We emphasise that this is an application-level observation on a deliberately simplified model, not a universal claim about biological control. In more complex or realistic models — or with different reward designs, action spaces, or RL algorithms — the relative value of cell versus substrate, and scalar versus image, may differ. PHYSIGYM is precisely the tool that makes such questions answerable in a reproducible way.

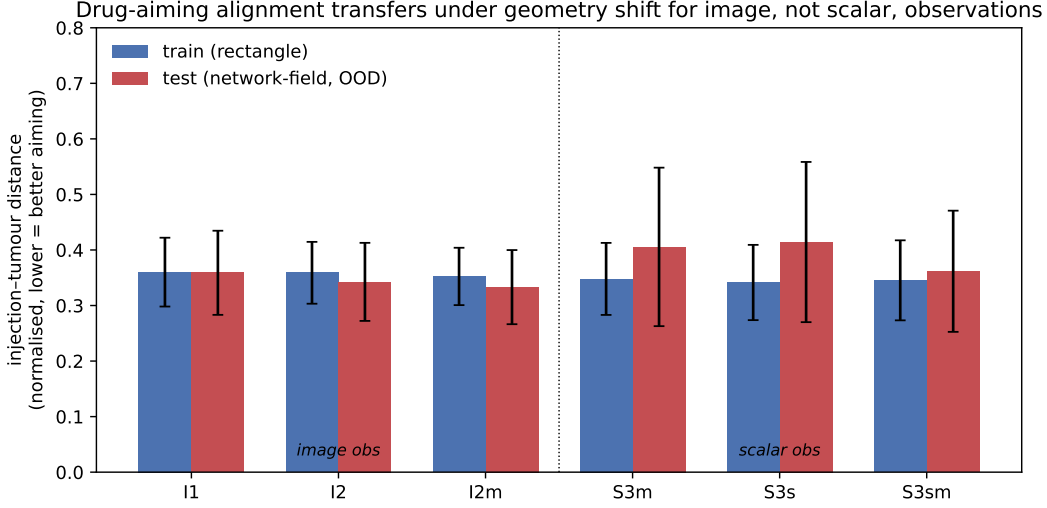


Figure 7: Drug-aiming alignment under geometry shift. Bars show the mean normalised distance between the drug-injection centre and the tumour centroid over dosed steps (lower is better aiming); error bars are the across-episode standard deviation. On the training rectangle (blue) image and scalar observations aim equally well. On the held-out network-field (red) image modes retain their aim while every scalar mode degrades with markedly higher variance, exposing the overfit as a failure to re-target the tumour rather than a loss of dosing ability.

Limitations. The toy model is intentionally small ($64 \mu\text{m}$, 2D) and cheap to simulate. Scaling to 3D or larger domains will increase the cost of image observations. All experiments use a single RL algorithm (SAC); alternative algorithms or model-based methods may interact differently with observation spaces. Each mode is trained with a modest, fixed number of seeds ($n = 5$); we report bootstrap confidence intervals (Appendix D) alongside raw seed-standard-deviation, but the conclusions would be strengthened by more seeds. We also study a single train/test layout direction (rectangle→network-field), chosen so that the structured layout is the training distribution and the irregular one is held out; evaluating the reverse direction (network-field→rectangle) is a natural control that we leave to future work, as it would confirm that the image advantage reflects transferable spatial structure rather than the test layout being intrinsically easier. Complex biological models and/or 3D models are left to future work.

5 Conclusion

We presented PHYSIGYM, an open-source framework that bridges the PHYSICELL ABM with Gymnasium RL, enabling direct policy learning in spatially explicit biological simulations. Using a TME model with an indirect drug mechanism, we showed that spatially explicit image observations are the critical state representation for *generalization*: training on a structured rectangle layout and testing on a held-out network-field, image modes retain $\approx +30$ to $+32$ test return while spatial-scalar summaries — competitive during training — collapse to ≈ 0 out of distribution, and a macrophage-count-only POMDP baseline fails to learn at all. The gap is a generalization phenomenon: what separates the representations is not training fit but whether the encoded structure transfers across spatial configurations. PHYSIGYM provides the infrastructure to study such questions in a principled, reproducible manner, and we anticipate it will serve as a testbed for both computational biology and RL communities.

References

S. Aif, M. Eiche, N. Appold, E. Fischer, T. Citak, and J. Kayser. Reinforcement failing guides the discovery of emergent spatial dynamics in adaptive tumor therapy. Apr. 2025. doi: 10.1101/2025.04.08.647768.

350 A. G. Barto, R. S. Sutton, and C. Watkins. *Learning and sequential decision making*, volume 89.
351 University of Massachusetts Amherst, MA, 1989.

352 R. Bellman. A markovian decision process. *Journal of mathematics and mechanics*, pages 679–684,
353 1957.

354 B. Chakraborty and S. A. Murphy. Dynamic treatment regimes. *Annual review of statistics and its
355 application*, 1(1):447–464, 2014.

356 L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley,
357 I. Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner
358 architectures. In *International conference on machine learning*, pages 1407–1416. PMLR, 2018.

359 A. Ghaffarizadeh, S. H. Friedman, and P. Macklin. Biofvm: an efficient, parallelized diffusive
360 transport solver for 3-d biological simulations. *Bioinformatics*, 32(8):1256–1258, 2016.

361 A. Ghaffarizadeh, R. Heiland, S. H. Friedman, S. M. Mumenthaler, and P. Macklin. PhysiCell: An
362 openb source physics-based cell simulator for 3-d multicellular systems. *PLOS Computational
363 Biology*, 14(2):e1005991, feb 2018. doi: 10.1371/journal.pcbi.1005991.

364 J. W. Gray and M. G. Pallavicini. Ara-c scheduling: Theoretical and experimental considerations.
365 *Medical and Pediatric Oncology*, 10(S1):93–108, 1982. ISSN 1096-911X. doi: 10.1002/mpo.
366 2950100711.

367 T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta,
368 P. Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*,
369 2018.

370 R. Heiland, D. Bergman, B. Lyons, G. Waldow, J. Cass, H. L. da Rocha, M. Ruscone, V. Noël, and
371 P. Macklin. Physicell studio: a graphical tool to make agent-based modeling more accessible.
372 *Gigabyte*, 2024:gigabyte128, 2024.

373 J. A. Johnson, D. R. Bergman, H. L. Rocha, D. L. Zhou, E. Cramer, I. C. Mclean, Y. W. Dance,
374 M. Booth, Z. Nicholas, T. Lopez-Vidal, A. Deshpande, R. Heiland, E. Bucher, F. Shojaeian,
375 M. Dunworth, A. Forjaz, M. Getz, I. Godet, F. Kurtoglu, M. Lyman, J. Metzcar, J. T. Mitchell,
376 A. Raddatz, J. Solorzano, A. Sundus, Y. Wang, D. G. DeNardo, A. J. Ewald, D. M. Gilkes, L. T.
377 Kagohara, A. L. Kiemen, E. D. Thompson, D. Wirtz, L. D. Wood, P.-H. Wu, N. Zaidi, L. Zheng,
378 J. W. Zimmerman, J. M. Phillip, E. M. Jaffee, J. W. Gray, L. M. Coussens, Y. H. Chang, L. M.
379 Heiser, G. L. Stein-O’Brien, E. J. Fertig, and P. Macklin. Human interpretable grammar encodes
380 multicellular systems biology models to democratize virtual cell laboratories. *Cell*, July 2025.
381 ISSN 0092-8674. doi: 10.1016/j.cell.2025.06.048.

382 R. Laubenbacher, F. Adler, G. An, F. Castiglione, S. Eubank, L. L. Fonseca, J. Glazier, T. Helikar,
383 M. Jett-Tilton, D. Kirschner, et al. Toward mechanistic medical digital twins: some use cases in
384 immunology. *Frontiers in Digital Health*, 6:1349595, 2024.

385 G. Letort, A. Montagud, G. Stoll, R. Heiland, E. Barillot, P. Macklin, A. Zinovyev, and L. Calzone.
386 PhysiBoSS: a multi-scale agent-based modelling framework integrating physical dimension and
387 cell signalling. *Bioinformatics*, 35(7):1188–1196, aug 2018. doi: 10.1093/bioinformatics/bty766.

388 X.-Y. Liu, D. Bodaghi, Q. Xue, X. Zheng, and J.-X. Wang. Asynchronous parallel reinforcement
389 learning for optimizing propulsive performance in fin ray control. *Engineering with Computers*, 41
390 (4):2201–2218, 2025.

391 J. Metzcar, Y. Wang, R. Heiland, and P. Macklin. A review of cell-based computational modeling in
392 cancer biology. *JCO Clinical Cancer Informatics*, pages 1–13, Dec. 2019. ISSN 2473-4276. doi:
393 10.1200/cci.18.00069.

394 V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu.
395 Asynchronous methods for deep reinforcement learning. In *International conference on machine
396 learning*, pages 1928–1937. PmLR, 2016.

397 V. Noël, M. Ruscone, R. Shuttleworth, and C. K. Macnamara. Physimess - a new physicell add-on
398 for extracellular matrix modelling. *Gigabyte*, 2024, Oct. 2024. ISSN 2709-4715. doi: 10.46471/
399 gigabyte.136.

400 K. O’shea and R. Nash. An introduction to convolutional neural networks. *arXiv preprint*
401 *arXiv:1511.08458*, 2015.

402 M. Ponce-de Leon, A. Montagud, V. Noël, A. Meert, G. Pradas, E. Barillot, L. Calzone, and
403 A. Valencia. Physiboss 2.0: a sustainable integration of stochastic boolean and agent-based
404 modelling frameworks. *npj Systems Biology and Applications*, 9(1):54, 2023.

405 M. L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley
406 & Sons, 2014.

407 Python Software Foundation. Extending and embedding the python interpreter. [https://docs.
408 python.org/3/extending/embedding.html](https://docs.python.org/3/extending/embedding.html), 2024. Accessed: 2024-02-07.

409 L. H. Rocha, B. Aguilar, M. Getz, I. Shmulevich, and P. Macklin. A multiscale model of immune
410 surveillance in micrometastases gives insights on cancer patient digital twins. *npj Systems Biology
411 and Applications*, 10(1), Dec. 2024. ISSN 2056-7189. doi: 10.1038/s41540-024-00472-z.

412 M. Ruscone, A. Checcoli, R. Heiland, E. Barillot, P. Macklin, L. Calzone, and V. Noël. Building
413 multiscale models with physiboss, an agent-based modeling tool. *Briefings in Bioinformatics*, 25
414 (6), Sept. 2024. ISSN 1477-4054. doi: 10.1093/bib/bbae509.

415 M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris,
416 M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis.
417 Gymnasium: A standard interface for reinforcement learning environments, 2024.

418 G. van Rossum. Extending and embedding the python interpreter. *CWI Report CS-R9527*, May 1995.
419 URL <https://gvanrossum.github.io/Publications.html>.

420 C. J. Watkins and P. Dayan. Q-learning. *Machine learning*, 8:279–292, 1992.

421 A. E. Zade, S. S. Haghighi, and M. Soltani. Reinforcement learning for optimal scheduling of
422 glioblastoma treatment with temozolomide. *Computer methods and programs in biomedicine*, 193:
423 105443, 2020.

424 A Generation of Initial Conditions

425 **Rectangle (training layout).** Each cell type is placed uniformly in a vertical strip of width 10% of
426 the x -axis, with non-overlapping strip positions sampled from $[0.1, 0.2]$, $[0.3, 0.5]$, $[0.6, 0.8]$ (shuffled
427 across types). This is a highly regular, low-entropy layout.

428 **Network-field (held-out test layout).** White Gaussian noise $\eta(\mathbf{x}) \sim \mathcal{N}(0, 1)$ is convolved with a
429 Gaussian kernel G_σ ($\sigma = \ell_c/\sqrt{2}$, $\ell_c = 45 \mu\text{m}$) to produce a correlated field F . An additional local
430 noise component $\tilde{\xi}$ filtered at width $\ell_c/3$ is added with small weight α to introduce local heterogeneity.
431 The field is normalised to $[0, 1]$ and cells are sampled from positions with $\hat{F} > 0.55$, with probability
432 proportional to $\hat{F}$, producing irregular, clustered configurations that are structurally distinct from the
433 training rectangle.

434 B Training algorithm and setup

435 To address our control objective — maximizing cancer-cell eradication while minimizing drug
436 administration — we employ Soft Actor-Critic (SAC) [Haarnoja et al., 2018], a state-of-the-art
437 RL algorithm for continuous control. SAC is well suited to our setting due to its high sample
438 efficiency, and as a deep RL method it accommodates different observation types, such as image-
439 based representations, through Convolutional Neural Networks (CNNs) [O’shea and Nash, 2015].
440 A key challenge is the computational cost of the simulator; to alleviate this bottleneck we adopt

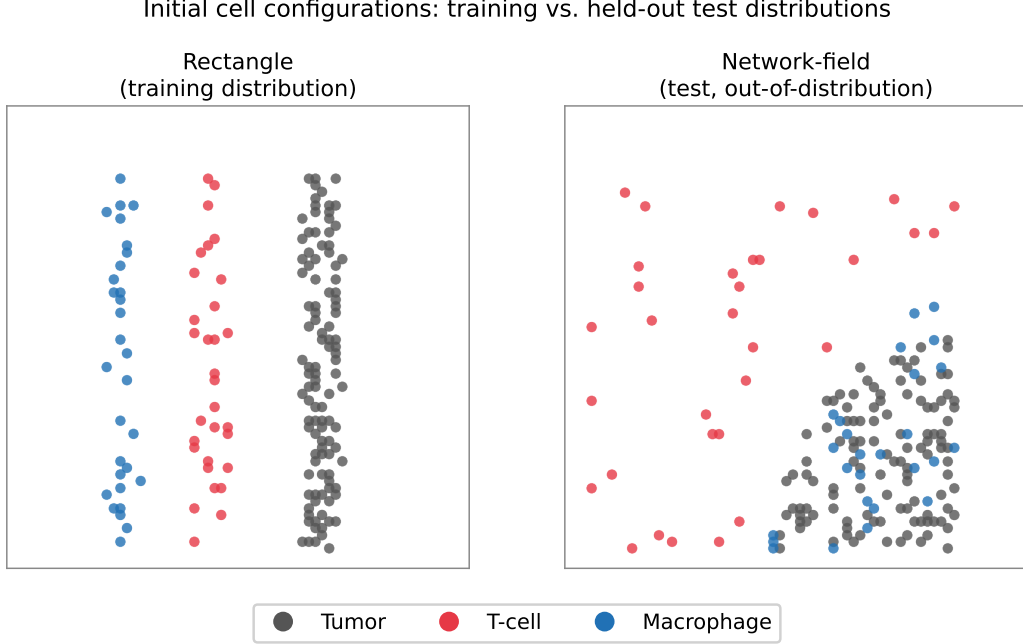


Figure 8: Representative examples of the training (rectangle) and held-out test (network-field) spatial layouts.

an asynchronous parallel training framework [Mnih et al., 2016, Liu et al., 2025]. A GPU-resident learner updates policy and value networks asynchronously while 13 CPU-resident environment processes generate experience in parallel.

Compute. Each run trains a single observation mode for one seed to 10^5 agent decisions (6×10^5 simulator steps) in roughly 3–5 hours of wall-clock time (including simulation, experience collection, and learner updates) on a single workstation (one NVIDIA GeForce RTX 4090 GPU-resident learner with 13 CPU-resident environment workers on an Intel Core i9-13900K, 32 logical cores). The asynchronous design decouples the GPU learner from the comparatively slow PHYSICELL simulator, so throughput is limited by the CPU environment workers rather than by gradient updates; this is what makes the full observation-space sweep (nine modes, 5 seeds each) tractable on a single machine.

Bridge overhead and throughput. To characterise the cost of the PHYSIGYM bridge itself we time a single environment’s `step()` in isolation (median over 60 steps after warm-up, one worker, on the workstation above); each `step()` runs the C++ PHYSICELL simulator step and the Python observation read-back that the bridge adds on top (Table 2). A single environment sustains ≈ 12 – 13 steps/s. Crucially, switching from the scalar POMDP observation to a full nine-channel image observation raises the per-step time by only $\approx 13\%$ ($76.9 \rightarrow 86.3$ ms), and the cell-only image is indistinguishable from the scalar mode: the observation read-back is a small fraction of the step, which is dominated by the simulator itself. This is why the choice of observation space (Section 2.4) barely affects wall-clock, and why aggregate throughput scales with the number of parallel CPU workers rather than with observation richness — the 13-worker configuration reaches on the order of ≈ 150 steps/s of aggregate experience, so the 6×10^5 -step budget plus learner updates completes in the 3–5 h per run reported above.

Action repeat. Each action selected by the agent is held fixed for 6 consecutive simulator steps (*action repeat* of 6) before the next decision. This serves two purposes. First, it stabilises learning: with a 15-minute decision interval on a slow, indirect biological process (drug injection $\rightarrow$ macrophage repolarisation $\rightarrow$ gradient shift $\rightarrow$ T-cell migration $\rightarrow$ tumor death), consecutive raw simulator steps are highly correlated and a fresh decision at every step injects high-frequency jitter into the drug field without changing the underlying dynamics; committing to an action for several steps lets

Table 2: Single-environment PHYSIGYM step cost (one CPU worker; median over 60 steps, warm-up discarded). The bridge’s observation read-back adds at most $\approx 13\%$ over the raw simulator step; image observations are not a throughput bottleneck.

Observation mode	ms/step (median)	p90 (ms)	steps/s
scalars_macrophages (POMDP)	76.9	78.5	13.0
img_mc_cells (I1)	75.6	77.6	13.2
img_mc_cells_substrates (I2)	86.3	88.3	11.6

its effect materialise before it is revised, which empirically yields smoother dose trajectories and lower-variance value targets. Second, it improves simulator throughput per decision: the 10^5 agent decisions per run correspond to 6×10^5 simulator steps of experience, so the effective interaction budget (and all step axes in Figures 4–6) is 6×10^5 . Action repeat is a standard device for temporally extended control and is well motivated here by the slow timescale of the controlled process relative to the simulator step. The specific value $AR=6$ is selected by a sensitivity sweep over $AR \in \{1, \dots, 8\}$ (Appendix B.4, Figure 9): the action-smoothness cost drops steeply and plateaus by $AR \approx 6$, and the composite control objective reaches its plateau over $AR=5-8$, so $AR=6$ is the smallest repeat that attains both.

B.1 Asynchronous SAC

Algorithm 1 Asynchronous SAC with Parallel Environments [Liu et al., 2025]

Require: Config d_{arg} , buffer size $|\mathcal{B}|$, LRs, γ , τ

- 1: Init shared queues `actor_queue`, `sample_queue`; stop signal `stop_event`
- 2: **Actor Process (CPU):** init $N_{\text{env}} = 13$ envs, local π_{θ_a}
- 3: **while** not `stop_event` **do**
- 4: Receive θ from `actor_queue` if available
- 5: Act: random if $t < T_{\text{start}}$, else π_{θ_a}
- 6: Step envs; push (s, a, r, s', d) to `sample_queue`
- 7: **end while**
- 8: **Learner Process (GPU):** init π_{θ} , $Q_{\phi_{1,2}}$, targets, $\mathcal{D}$
- 9: **while** total samples $< T$ **do**
- 10: Drain `sample_queue` into $\mathcal{D}$
- 11: **for** N_{loops} gradient steps **do**
- 12: Sample minibatch; compute SAC target y ; update Q , π , α ; soft-update targets
- 13: **end for**
- 14: Push updated θ to `actor_queue`
- 15: **end while**

B.2 Neural Architecture

Scalar modes utilize a Multi-Layer Perceptron (MLP):

$$\text{FC}(256) \xrightarrow{\text{Mish}} \text{FC}(256)$$

Image modes utilize an IMPALA-style CNN [Espeholt et al., 2018]. The feature extraction and projection pipeline is defined as:

$$\begin{aligned} & \text{Conv}(n_c \rightarrow 32, k=8 \times 8, s=4) \xrightarrow{\text{Mish}} \\ & \text{Conv}(32 \rightarrow 64, k=4 \times 4, s=2) \xrightarrow{\text{Mish}} \\ & \text{Conv}(64 \rightarrow 64, k=3 \times 3, s=1) \xrightarrow{\text{Mish}} \text{Flatten} \rightarrow \text{FC}(256) \rightarrow \text{FC}(256) \end{aligned}$$

Table 3: SAC hyperparameters (TME v2 experiments).

Parameter	Symbol	Value
Agent decisions	T	1×10^5
Action repeat	—	6
Effective simulator steps	$6T$	6×10^5
Replay buffer size	$ \mathcal{B} $	2×10^5
Batch size	B	832
Warm-up steps	T_{start}	5,000
Policy / critic LR	η_π, η_Q	3×10^{-4}
Discount factor	γ	0.99
Target smoothing	τ	0.005
Reward weights	$w_{\text{cell}}, w_{\text{dose}}$	0.3, 2.0
Entropy autotuning	—	enabled
Parallel envs	N_{env}	13
Test frequency	—	every 4 episodes

483 B.3 Hyperparameters

484 B.4 Choice of action repeat

485 We select the action-repeat value with a dedicated sensitivity sweep, run independently of the main
 486 observation-space experiments. We sweep `action_repeat` $\in \{1, \dots, 8\}$ and, for each value, measure
 487 three end-of-run quantities aggregated over repeated rollouts (with 95% bootstrap CIs): tumor
 488 reduction (control efficacy), an action-smoothness cost $\sum_t \|a_t - a_{t-1}\|^2$ (policy jitter; lower is
 489 better), and a composite objective combining tumor reduction, dose, and smoothness (Figure 9).

490 Two effects drive the choice. (i) *Smoothness*: the action-smoothness cost falls steeply and monoton-
 491 ically with action repeat — from ≈ 110 at AR= 1 to ≈ 17 at AR= 6 — and then flattens, so most
 492 of the stabilisation benefit is already captured by AR= 6 and larger values add little. This matches
 493 the physical timescale of the task: with a 15-minute decision interval on a slow immune process,
 494 re-deciding every simulator step injects high-frequency drug-field jitter that does not change the
 495 underlying dynamics. (ii) *Composite objective*: the objective is worst at AR= 1 (-0.88) and rises
 496 to a plateau over AR= 5–8, with AR= 6 at -0.07 ; within that plateau the values are within each
 497 other’s confidence intervals, so AR= 6 is chosen as the smallest repeat that reaches the objective
 498 plateau while sitting at the knee of the smoothness curve. This makes action repeat a deliberate,
 499 evidence-backed choice rather than an incidental hyperparameter.

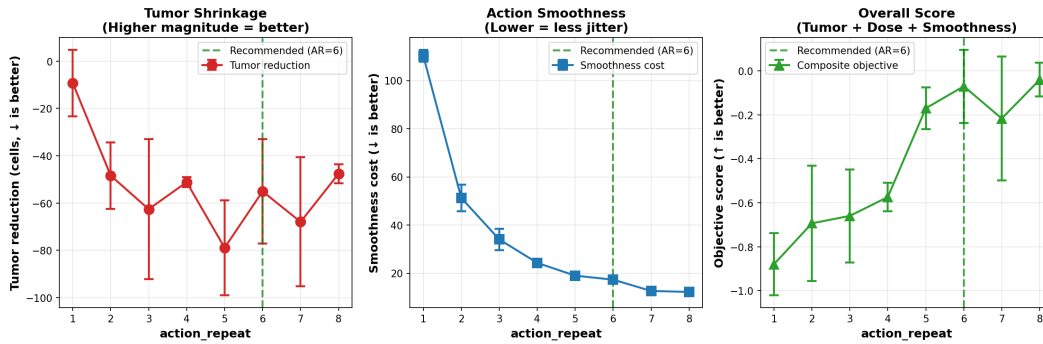


Figure 9: Action-repeat sensitivity sweep (independent of the observation-space experiments; error bars are 95% bootstrap CIs). **Left:** tumor reduction (more negative = more cells removed). **Centre:** action-smoothness cost — drops sharply and plateaus by AR $\approx$ 6. **Right:** composite objective (higher is better) — rises to a plateau over AR= 5–8; AR= 6 (dashed) is the smallest repeat on that plateau and at the knee of the smoothness curve.

C Full observation-mode comparison

Table 4 reports all nine observation modes evaluated, including the m1m2 scalar variants omitted from the curated main-text Table 1. Train and test returns are end-of-training EWMA-50 means with the standard deviation across seeds; train = rectangle, test = network-field.

Table 4: All observation modes (seeds averaged; train = rectangle, test = network-field). Modes are ordered by held-out test return.

ID	Mode	n	Train $_{\mu}$	Train $_{\sigma}$	Test $_{\mu}$	Test $_{\sigma}$
I1	img_mc_cells	5	+30.8	9.5	+33.4	11.3
I2	img_mc_cells_substrates	5	+45.3	5.0	+32.7	4.7
I2m	img_mc_cells_substrates_m1m2	5	+40.8	8.5	+32.5	9.4
I1m	img_mc_cells_m1m2	5	+35.0	13.1	+26.0	12.4
S5m	spatial_scalars_cells_spatial_no_scalars_substrates_m1m2	5	+53.6	34.2	+9.5	9.6
S3sm	spatial_scalars_cells_substrates_m1m2	5	+38.3	27.8	+2.6	12.1
S3m	spatial_scalars_cells_m1m2	5	+41.8	27.7	+2.5	10.6
S3s	spatial_scalars_cells_substrates	5	+42.9	18.8	−0.0	9.1
RAND	random policy (baseline)	5	−18.4	4.0	−25.3	3.0
POMDP	scalars_macrophages	5	−20.4	8.3	−31.8	2.3

D Bootstrap confidence intervals

Because each mode is trained with only $n = 5$ seeds, the across-seed standard deviations reported in Tables 1 and 4 are themselves noisy estimates of dispersion. To quantify uncertainty on the *mean* return more robustly, we bootstrap over seeds: for each mode we resample the seeds with replacement $B = 10,000$ times, recompute the mean end-of-training return of each resample, and report the 2.5th–97.5th percentiles of that bootstrap distribution as a 95% confidence interval on the mean (Table 5). The same procedure applied per step yields the shaded bands in Figures 10 and 11.

The bootstrap intervals sharpen the main-text claims. On the held-out network-field, the three image modes have test-return CIs that lie entirely above +24, whereas every spatial-scalar mode has a test-return CI that *contains zero* (e.g. S3s [−6.2, +9.1]), so their generalization is not statistically distinguishable from a non-learning policy. The image and spatial-scalar CIs do not overlap, confirming that the generalization gap is significant and not an artefact of seed selection. The scalars_macrophages POMDP baseline has a tight, strongly negative CI [−33.4, −29.4], reflecting consistent failure rather than robustness — and it does not overlap the random policy’s test CI [−28.2, −22.8] from the favourable side, so the macrophage-only observation performs *no better than, and plausibly worse than, random dosing*.

Table 5: Mean end-of-training return with a 95% bootstrap confidence interval on the mean ($B = 10,000$ seed resamples; train = rectangle, test = network-field). Ordered by test mean.

ID	Mode	n	Train $_{\mu}$ (95% CI)	Test $_{\mu}$ (95% CI)
I1	img_mc_cells	5	+30.8 [+21.3, +38.0]	+33.4 [+23.6, +43.2]
I2	img_mc_cells_substrates	5	+45.3 [+40.5, +49.4]	+32.7 [+28.6, +36.8]
I2m	img_mc_cells_substrates_m1m2	5	+40.8 [+33.0, +47.3]	+32.5 [+24.4, +40.6]
I1m	img_mc_cells_m1m2	5	+35.0 [+22.8, +45.6]	+26.0 [+14.1, +37.9]
S5m	spatial_scalars_cells_spatial_no_scalars_substrates_m1m2	5	+53.6 [+25.0, +85.1]	+9.5 [+0.4, +17.6]
S3sm	spatial_scalars_cells_substrates_m1m2	5	+38.3 [+20.1, +65.9]	+2.6 [−6.2, +14.3]
S3m	spatial_scalars_cells_m1m2	5	+41.8 [+19.8, +68.0]	+2.5 [−7.6, +11.6]
S3s	spatial_scalars_cells_substrates	5	+42.9 [+24.7, +58.8]	−0.0 [−6.2, +9.1]
RAND	random policy (baseline)	5	−18.4 [−21.9, −14.9]	−25.3 [−28.2, −22.8]
POMDP	scalars_macrophages	5	−20.4 [−28.6, −13.8]	−31.8 [−33.4, −29.4]

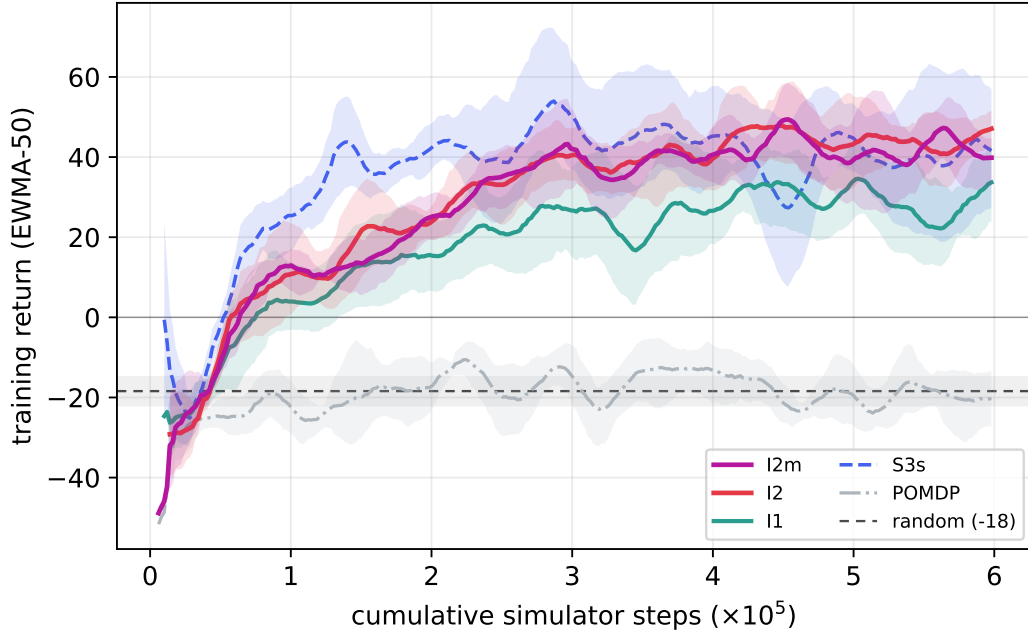


Figure 10: Training return on rectangle (EWMA-50, mean with 95% bootstrap CI band over seeds) vs. cumulative simulator steps. Curated main modes only.

E Significance tests and effect sizes

The bootstrap CIs (Appendix D) quantify uncertainty on each mode’s mean; here we add explicit tests so the main claims do not rest on visual CI overlap alone. All tests use the same per-seed end-of-training EWMA-50 test returns ($n = 5$ per mode) and introduce no new runs.

Image vs. spatial-scalar generalization. Pooling the three image modes’ per-seed test returns ($n = 15$, mean $+32.8$) against the four spatial-scalar modes’ ($n = 20$, mean $+3.7$), a one-sided Mann–Whitney U -test gives $U = 296$, $p = 6.2 \times 10^{-7}$, and Cliff’s $\delta = +0.97$ — near-complete rank separation between the families. This is the formal counterpart to the disjoint CIs in Table 5.

M1/M2-split ablation (equivalence, not just non-significance). For the two nested pairs we report both the difference-of-means test and a two-one-sided-tests (TOST) equivalence test against a margin of ± 10 ($\approx 1\sigma$ of a single mode’s seed spread). I1 vs. I1m: $+33.4$ vs. $+26.0$, difference not significant (Welch $p = 0.40$) but TOST does *not* establish equivalence ($p = 0.38$). I2 vs. I2m: $+32.7$ vs. $+32.5$, Welch $p = 0.97$, TOST $p = 0.05$ (borderline). At $n = 5$ the data therefore support “no detectable improvement from the pre-computed split” but do not license the stronger claim of statistical equivalence; more seeds would be needed to settle it.

Seed-variance across families. A Levene test (median-centred) on the across-seed test-return spread of the image versus spatial-scalar families is not significant ($W = 0.67$, $p = 0.42$), consistent with the main text’s statement that no family shows a systematic seed-variance advantage on this benchmark.

The numbers above are produced by `figures_plotting/stats_tests.py`, which reads the same cached per-seed curves as the tables and figures.

F Example Episode Comparisons

Figure 12 shows a single matched test episode comparing an image mode (`img_mc_cells_substrates`), a spatial-scalar mode (`spatial_scalars_cells_substrates`),

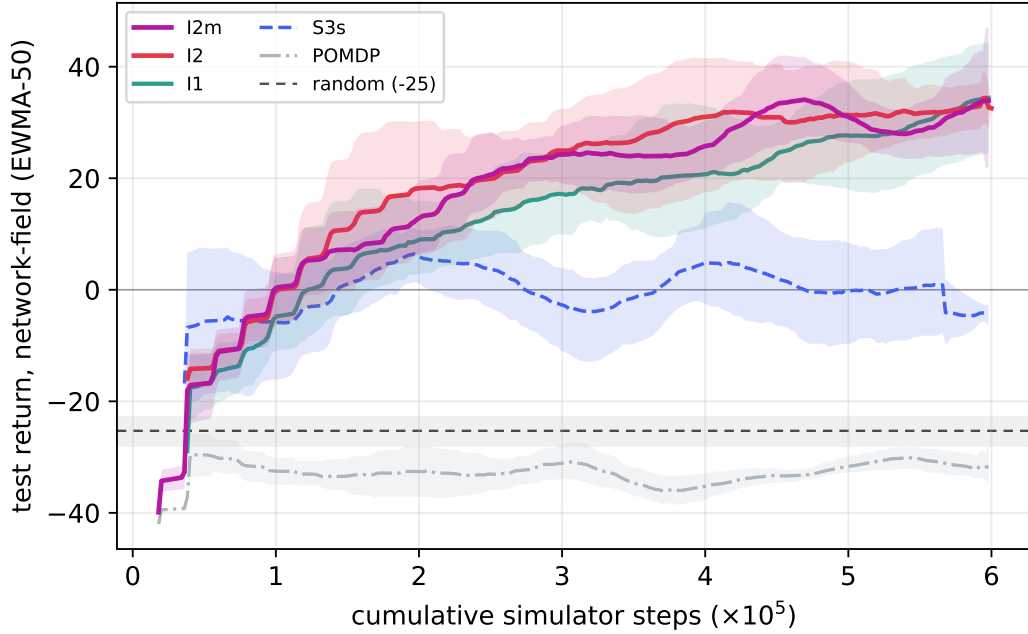


Figure 11: Held-out test return on network-field (EWMA-50, mean with 95% bootstrap CI band over seeds) vs. cumulative simulator steps. The image modes’ bands separate from the spatial-scalar summary (S3s), whose band hugs zero, and from the POMDP baseline.

544 and the POMDP baseline (`scalars_macrophages`) step by step. To guarantee an exact comparison,
545 all three agents are rolled out (deterministically, using the policy mean action) from the *identical*
546 held-out network-field initial condition — the same 124-tumor-cell layout replayed from file — so
547 the only difference between the three trajectories is the observation mode.

548 The episode cleanly separates the three regimes. `img_mc_cells_substrates` drives the tumor
549 to near-eradication (3 cells remaining, +223 cumulative return); it spends the most drug (4.5
550 total dose) but places it precisely enough — visibly dosing in sustained, targeted bursts (Fig-
551 ure 12, bottom) — to convert dose into effective macrophage repolarisation and tumor clearance.
552 `spatial_scalars_cells_substrates` achieves partial control (70 cells, +30 return, 2.8 dose):
553 it suppresses growth early but plateaus and cannot finish, lacking the spatial substrate feedback
554 needed to target the final push. `scalars_macrophages` (POMDP) barely acts (0.9 total dose) and
555 the tumor *grows* past its starting size (130 cells, −6.4 return): seeing only macrophage-polarisation
556 counts, it cannot localise where to inject, so it under-doses and fails — consistent with its aggregate
557 performance at or below the random baseline.

Matched network-field episode (identical IC, 124 tumor cells)

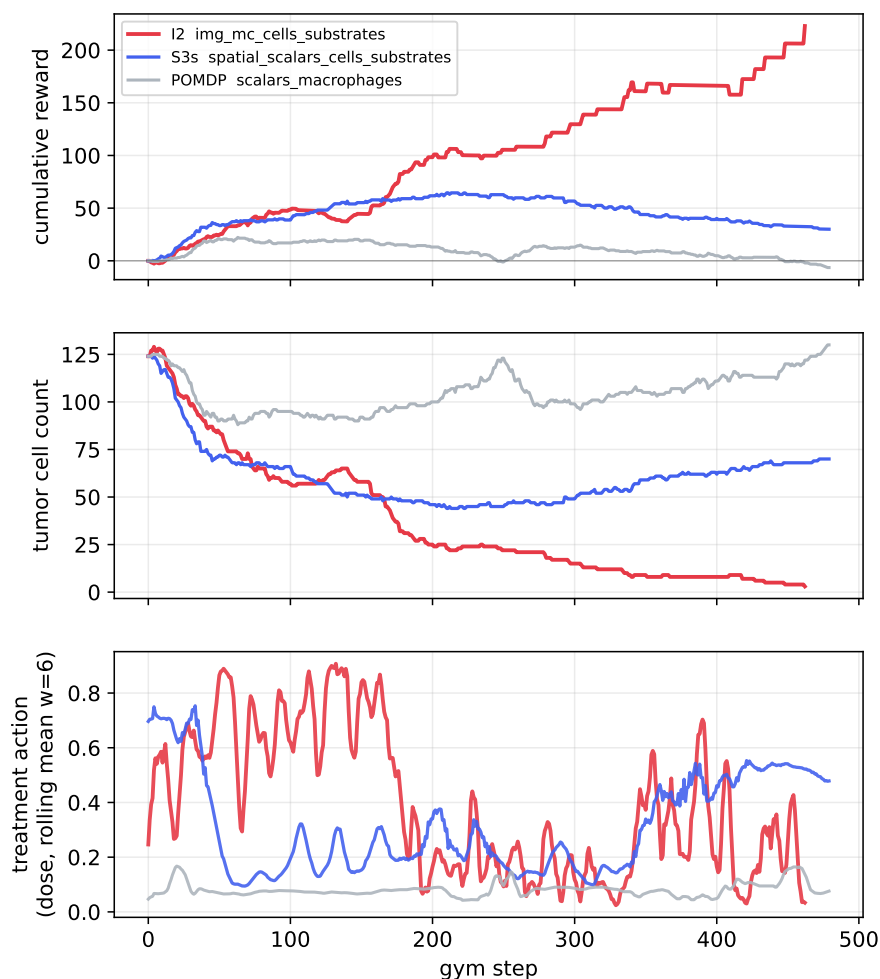


Figure 12: **Matched network-field episode (identical initial condition, 124 tumor cells)**. Cumulative reward (top), tumor cell count (middle), and the dose action (bottom, rolling mean over 6 steps — matching the action-repeat horizon used in training; the raw per-step signal is noisy and unreadable at full resolution) for `img_mc_cells_substrates` (red), `spatial_scalars_cells_substrates` (blue), and the `scalars_macrophages` POMDP baseline (grey), all replayed from the same IC. `img_mc_cells_substrates` reaches near-eradication (3 cells, +223 return, 4.5 dose); `spatial_scalars_cells_substrates` achieves partial control (70 cells, +30 return, 2.8 dose); the POMDP baseline under-doses and the tumor grows (130 cells, −6.4 return, 0.9 dose). Only the image mode sustains a high, decisive dose action long enough to drive the tumor to near-eradication, whereas `spatial_scalars_cells_substrates` oscillates and eventually lets the tumor regrow. Note that the physical dose delivered to the simulation each step is the concentration action (bottom) multiplied by the voxel volume swept by the independently-controlled injection radius action, so the dose action alone does not linearly predict the physical dose spent.